DAV 6150 Module 6: Regression Modeling for Numeric Response Variables

Module 5 Assignment Review

A Simple Method for Calculating a ROC Curve

A ROC curve is formulated by plotting the **true positive rate** (TPR) (a.k.a., **sensitivity**) against the **false positive rate** (FPR) (a.k.a, **1 - specificity)** for a given classifier's actual classifications and scored probabilities.

The FPR is represented by the X axis while the TPR is represented by the Y axis.

Remember:

- The **true positive rate** (TPR) is the proportion of actual "positive" classification values that are **correctly identified as "positive"**. This is also known as the **sensitivity** of a classifier, i.e., TP / (TP + FN)
- The **false positive rate** (FPR) is the proportion of actual "negative" classification values that are **incorrectly identified as "positive"**. FPR is defined as FP / (FP + TN), which is the same as **1 - specificity** (remember that specificity = TN/(TN + FP))

To derive the values required for a ROC plot of a model we **iterate through an ordered sequence of 'K' classification threshold values** within the range of $(0 < threshold <= 1)$ (e.g., if 'K' == 100, our iterator would span .01 through 1.00 via .01 increments).

While iterating through the threshold values, we **compare the scored probabilities against each iterative threshold value** to derive + store TPR and FPR values across the full range of iterable threshold values. When finished, we have accumulated 'K' pairs of TPR and FPR values.

Then, plot the resulting TPR and FPR metrics against one another on a 2-dimensional plane.

A Python-based example

```
''' PLEASE NOTE THAT THIS PYTHON CODE IS SHOWN ONLY FOR PURPOSES OF EXPLAIN
HOW TO APPROACH THE CREATION OF A ROC CURVE FOR A BINARY CLASSIFIER
TO MAKE USE OF IT YOU WOULD NEED TO INCORPORATE IT WITHIN YOUR OWN CONTEXT

'actual' is the class variable (the actual classification) from the data se
'probability' is the scored.probability variable from the data set '''

# --------------------------------------------------------------------

    #create list of thresholds
    thresholds = np.arange(0,1,0.01)

    #create list of true positive rates
    tpr_list=[]

    #create list of false positive rates
    fpr_list=[]

    #create new list of predictions for every threshold
    for i in thresholds: #for every threshold
        new_predicted=[] #create new list of predictions
        for j in range(len(probability)): #for each observation
            #compare threshold to probability and append new prediction
            if probability[j] < i:
                new_predicted.append(0)
            else:
                new_predicted.append(1)

        #calculate the tpr
        tpr = sensitivity(actual, pd.Series(new_predicted))
        tpr_list.append(tpr) #append the true positive rate to the list

        #calculate the fpr
        fpr = 1 - specificity(actual, pd.Series(new_predicted))
        fpr_list.append(fpr) #append the false positive rate to the list

# ...then continue to formulate your code for plotting tpr_list vs. fpr_lis
# via the Python graphics tool of your choice
```

Calculating AUC

Once you have derived the data values required for plotting a ROC curve, calculating the area under that curve is a simple matter of applied mathematics / calculus. There are a variety of mathematical approaches for estimating the area under a curve, with two of the most prominent being:

- The **trapezoidal rule**: https://www.khanacademy.org/math/ap-calculus-ab/ab-integration-new/ab-6-2/a/understanding-the-trapezoid-rule
- **Riemann sums** (a.k.a., the "rectangular" rule): https://www.khanacademy.org/math/ap-calculus-ab/ab-integration-new/ab-6-2/a/riemann-sums-review.

Regression Modeling for Numeric Response Variables

In general, the phrase **regression modeling** refers to the process of estimating the strength of the relationship between one variable (the **"Response" (aka "Dependent") variable**) and one or more **"Explanatory" (aka "Independent") variables**.

Regression modeling is widely used for purposes of **predicting the mean numerical value of the response variable** when given specific values for the explanatory variables.

There are many different types of regression models, and the characteristics of each vary depending on the type of response variable you are attempting to estimate. Some of the most widely used types of regression models for **numerical response variables** include:

- **Linear Regression**: Used for fitting a linear equation for a **continuous** numeric response variable. The relationship between the response and explanatory variable(s) is assumed to be **linear** in nature. The output is a **linear equation**. If there is only one explanatory variable, we use **Simple Linear Regression**. If there is more than one explanatory variable, we use **Multiple Linear Regression**.
- **"Count" Regression**: The response variable contains **non-negative** (i.e., x >= 0) **discrete** numeric "count" values while the explanatory variables can be either binary, discrete or continuous. Commonly used regression techniques for modeling count data include **Poisson Regression** (where the response variable **must** have a Poisson distribution), **Negative Binomial Regression** (where the response variable need not have a Poisson distribution), and **Zero-Inflated Negative Binomial Regression** (used when the response variable contains an excessive number of 'zero' values).
- **Polynomial Regression**: Used when a **non-linear** relationship exists between a continuous response variable and explanatory variables that are either binary, discrete or continuous. The ouput is a **non-linear** equation.

** Please note that there are **many** other types of regression models (e.g., Lasso, Ridge, Partial Least Squares, etc.), and describing each of them is beyond the scope of this

lecture.**

Linear Regression

Linear regression consists of finding the best fitting straight line through a set of data points. That line is called a **regression line**. An example:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

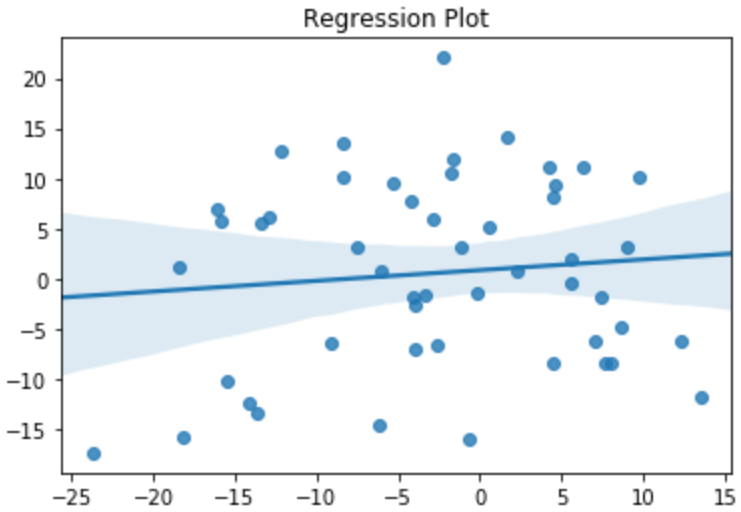
```
# generate some random data to use for the plot
x = np.random.randn(50) * 10

y = np.random.randn(50) * 10

# first define a Matplotlib figure
plt.figure()

# use Seaborn's regplot() function to plot the regression line for the
sns.regplot(x, y)

# give the plot a title using Matplotlib
plt.title('Regression Plot');
```



Simple linear regression equations take the form of $y' = bx + a$ where y' is the predicted value of the response variable for a given value of the explanatory variable x, b is the slope of the regression line, x is the value of the explanatory variable and a is a constant representing the y-intercept of the regression line.

The regression line is created by using the known values of the explanatory variable x and the known values of the response variable y to find the slope b and y-intercept a of the line that minimizes the sum of the squared errors of the predicted values of y'.

We calculate values for the slope b and the y-intercept a using the means and standard deviations of the known values of x and y as well as the correlation coefficient r of x and y:

$b = r * stddev(y)/stddev(x)$

$a = mean(y) - b(mean(x))$

NOTE: The Python modeling libraries we are discussing below do these calculations for you. The calculations are shown here simply to provide further context.

A simple linear regression example:

http://onlinestatbook.com/2/regression/intro.html

Multiple Linear Regression

When using multiple explanatory variables, the regression line **cannot be visualized within a two-dimensional space**. However, it is still relatively easy to compute: the regression line is estimated using the formula

$y' = a + b1x1 + b2x2 + . . + bnxn$

where y' is the predicted value for the response variable for given values of the explanatory variables x1, x2, .., xn, and the b values represent the **independent contributions** of the corresponding explanatory variable to the prediction of the response variable.

For those who desire further explanation of the underlying mathematics:

http://faculty.cas.usf.edu/mbrannick/regression/Reg2IV.html

Count Regression

Unlike linear regression, **count regression** is appropriate for models in which the response variable is a **non-negative integer**. The response variable should follow either a **Poisson distribution** (i.e., mean == variance) or an **overdispersed distribution** (i.e., variance > mean).

Why not just use some form of linear regression for estimating non-negative integers?

- The relationship between the explanatory variables and the non-negative integer response variable **are unlikely to be linear**.
- Linear regression can produce **negative floating point numbers**, which are clearly not valid if the response variable is known to be a non-negative integer.
- The distribution of the residuals from a linear regression model that has been applied to a non-negative integer response variable **will not be random**.

Poisson Regression

If your data follows a Poisson distribution, **Poisson Regression** can be an effective tool for estimating the non-negative integer response variable. The probability mass function (PMF) for a Poisson distribution is:

$$p(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

where k is any non-negative integer, e is the natural exponent $e = 2.71828$, and λ is the mean value of the response variable. (Note that the mean is equivalent to the variance in a Poisson distribution). Using the PMF we can estimate the likelihood of any value within a Poisson distribution.

Poisson regression incorporates this PMF as it calculates a **Maximum Likelihood Estimation** for the regression coefficients that maximize the likelihoods of the individual response variable values.

The output of the algorithm will be the **natural log** of the count value you are attempting to estimate. Therefore, you must **exponentiate** the output of the model to determine the actual estimated values for the count variable.

How to interpret the model's coefficients: "For a one unit change in X_k, the estimated count changes by a factor of $exp(\beta_k)$, assuming all other variables are held constant." However, in practice it is very common to simply examine the **directionality** of the coefficients, i.e., if a coefficient is **negative**, then the larger the value of the explanatory variable, the more it will tend to **decrease** the magnitude of the response variable. Conversely, if a coefficient is **positive**, then the larger the value of the explanatory variable, the more it will tend to **increase** the magnitude of the response variable.

See this article from the assigned readings for more detail on the underlying mathematics + an example of how to implement Poisson regression in Python: https://timeseriesreasoning.com/contents/poisson-regression-model/

Negative Binomial Regression

If your data is **over dispersed** (i.e, variance > mean), **Negative Binomial Regression** can be an effective tool for estimating the non-negative integer response variable. The probability mass function (PMF) for a Negative Binomial ("overdispersed") distribution is:

$$p(X = n) = \frac{(n-1)!}{(r-1)!(n-r)!} p^r (1-p)^{(n-r)}$$

where n is the number of attempts, r is the number of successes, and p is the probability of success. For our purposes "probability of success" represents the likelihood that the response variable has been observed having a specific value (e.g., 0 or 1 or 2 or ...).

Negative Binomial regression incorporates this PMF as it calculates a **Maximum Likelihood Estimation** for the regression coefficients that maximize the likelihoods of the individual response variable values.

As with Poisson regression, the output of the negative binomial regression algorithm will be the **natural log** of the count value you are attempting to estimate. Therefore, you must **exponentiate** the output of the model to determine the actual estimated value for the count variable.

How to interpret the model's coefficients: Interpretation of the explanatory variable coefficients is identical to Poisson regression: "For a one unit change in X_k, the estimated count changes by a factor of $exp(\beta_k)$, assuming all other variables are held constant." However, in practice it is very common to simply examine the **directionality** of the coefficients, i.e., if a coefficient is **negative**, then the larger the value of the explanatory variable, the more it will tend to **decrease** the magnitude of the response variable. Conversely, if a coefficient is **positive**, then the larger the value of the explanatory variable, the more it will tend to **increase** the magnitude of the response variable.

See this article from the assigned readings for an explanation of how to implement negative binomial regression in Python: https://timeseriesreasoning.com/contents/negative-binomial-regression-model/

See this article from the assigned readings for an explanation of how to implement **both** Poisson and Negative Binomial Regression via **Generalized Linear Models** (GLM's: https://en.wikipedia.org/wiki/Generalized_linear_model) using Python's **statsmodels** library:

- https://dius.com.au/2017/08/03/using-statsmodels-glms-to-model-beverage-consumption/#cameron

This article provides a detailed tutorial of how to implement both Poisson and Negative Binomial regression using Generalized Linear Models (GLM's) via Python's statsmodels library. **DAV 6150 students should strongly consider making use of this tutorial when attempting to implement a count regression model within Python**.

Polynomial Regression

When a **non-linear** relationship exists between a continuous response variable and explanatory variables that are either binary, discrete or continuous, a linear regression model is highly unlikely to be an effective model. As an alternative, we can substitute a **polynomial equation** for the standard linear model. The ouput of such a regression model is a **non-linear** equation.

What is a non-linear equation? An example:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \ldots + \beta_d x_i^d + \epsilon_i$$

As d (the degree of the polynomial) increases, the "flexibility" of the regression model increases significantly. See the assigned reading for an example: https://medium.com/towards-data-science/polynomial-regression-bbe8b9d97491

HOWEVER, this flexibility comes at a cost: we can easily end up with a model having very high variance (i.e., the model **overfits** the data). A model with very high variance is unlikely to prove to be effective when applied to previously unseen data.

THEREFORE: when using polynomial regression we need to be acutely aware of the **bias vs variance** tradeoff and work diligently to select a polynomial degree d that strikes a balance between the two.

It is unusual to use a d of more than 3 or 4 since using larger values can easily result in overfitting. However, there is no strict limitation on the selection of a value for d so use your empirical skills to help you decide on a value for d that is best suited to the data you are working with.

Evaluating Regression Model Performance

In **Module 5** we discussed the following regression model performance metrics:

- R^2: Often referred to as "Goodness of fit"; measures how much of the variation in the response variable y' is explained by variation in the explanatory variable(s). In general, **the larger the value of R^2, the more accurate the model is**. However, relatively large values of R^2 can also be an indication of the model being "overfit" to the training data.
- **Adjusted R^2**: Increases if a new variable added to the model improves the fit of the model by more than would be expected by sheer chance. When comparing two

models derived from the same data, models with **higher Adjusted** R^2 scores are preferable to those having relatively lower scores.

- **AIC**: Akaike Information Criteria is a model selection metric that estimates the relative quality of a statistical model for a given set of data. When comparing two models derived from the same data, models with **lower AIC** scores are preferable to those having relatively higher scores.
- **BIC**: Bayesian Information Criteria is another model selection metric that estimates the unexplained variation in the response variable relative to the given explanatory variables. **BIC** also imposes a "complexity" penalty when the number of explanatory variables used is increased. When comparing two models derived from the same data, models with **lower BIC** scores are preferable to those having relatively higher scores.
- **F Statistic**: Indicates whether a significant amount of variance in the response variable y' is explained by the model. When comparing two models derived from the same data, models with **higher F Statistic** scores are preferable to those having relatively lower scores.
- **Log Likelihood**: A measure of how well a model fits the underlying data. When comparing two models derived from the same data, models with **higher Log Likelihood** scores are preferable to those having relatively lower scores.
- **p values**: Measure the statistical significance of the explanatory variables in your model. While you are free to select the significance level on your own, most often 0.05 is used as the maximum bound for significance. As such, if any variable in your model is shown to have a **p value** that exceeds 0.05, consider removing it from the model to see whether the fit/model selection metrics improve.
- **Root Mean Squared Error (RMSE)**: Average distance of a sample from its observed value to its predicted value. We calculate the RMSE by finding the square root of the average of the squared values of a model's residual values. When comparing two models derived from the same data, models with **lower RMSE** scores are preferable to those having relatively higher RMSE scores.

These metrics allow us to assess the efficacy of our regression models. Practitioners should examine such metrics whenever they are readily available (e.g., if the software being used generates them automatically). If such metrics are not readily available as output from a model you are testing, you should consider whether your work requires you to calculate them via some other method as a separate step during your model evaluation work.

Residual Plots & Examining the "Normality" of Residuals

In addition to the metrics listed above, we can also make use of the **residuals** of a regression model to help us determine whether or not our models are robust.

A **residual** is the difference between the actual value of a response variable for a given observation and the predicted response variable value for that same observation.

In general, the residuals for **linear regression models** should be **normally distributed** and **homoscedastic** (i.e., their distribution is random across all estimated values and no pattern is emminently obvious). We can assess the residuals of a linear regression model via various types of plots. From your assigned readings (MLPR Ch 15): https://github.com/mattharrison/ml_pocket_reference/blob/master/ch15.ipynb

Note that non-linear models are unlikely to have normally distributed residuals; as such, testing for the normality of residuals of such models is unnecessary.

Project 1 Guidelines / Requirements